

JS Array

1. Introduction to Arrays in JavaScript

- a. An array in JavaScript is a *special type of object* used to store multiple values in a single variable.
- b. In simple words we can say that, array is the collection of data which may be homogenous or heterogeneous.
- c. **Types of Arrays:**

- i. **Homogenous Array:** The array which contains the same type of elements.

For Example → All Numbers, All Strings, etc.

```
let arrNum = [1, 2, 3, 4, 5]; // Numbers array
let arrStr = ["hello", "bye"]; // Strings
```

Array

- ii. **Heterogenous Array:** The array which contains the different types of elements.

For Example → Combination of numbers, strings, function, objects, etc

```
let mixedArr = [10, "hello", true,
{key:"value"}, function() {return "bye"}}];
```

2. Ways to create Arrays:

- a. We can create array in main two ways:
 - i. By Using Array Literal
 - ii. By Using Array Constructor

1. By Using Array Literal: (`[]`)

- a. The most common and straightforward way to create arrays is by using array literals.

- b. **Syntax** →

```
 let arrayRefVar = [value1, value2, value3, ..., valueN];
```

- c. **Accessing Elements of an Array:** You can access elements using their index (zero-based).

```
let arrStr = ["apple", "banana", "cherry"];  
console.log(arrStr[0]); // apple
```

2. By Using Array Constructor: (`new Array()`)

- a. The Array constructor allows you to create arrays dynamically.

- b. **Syntax** →

```
 let arrayRefVar = new Array(element1, element2, ..., elementN);
```

- c. **For Example** →

```
let arr = new Array(1, 2, 3);  
console.log(arr); // (3) [1, 2, 3]
```

- d. **Special Case: Passing a Single Number to the Constructor**

If you pass a *single number* as an argument to the Array constructor, it will be treated as the *length* of the array, not an element.

For Example →

```
let arr = new Array(5);  
console.log(arr); // (5) [empty × 5]
```

3. Array Property:

a. **length** property in Array:

- i. The length property of an array in JavaScript returns or sets the number of elements in the array.
- ii. It is a **read/write** property, meaning you can also modify it to resize the array.
- iii. Syntax → **array.length**

iv. **Example-1:** Getting the Length of an Array:

```
let arr = ["john", "smith", "dave"];  
console.log(arr.length); // 3
```

v. **Example-2:** Setting the Length to Truncate an Array:

```
let arr = ["john", "smith", "dave"];  
arr.length = 2;  
console.log(arr); // (2) ['john', 'smith']
```

4. Array Static Methods:

- a. Static methods of Array are built-in helper functions that we call directly on the **Array** class, not on array objects.
- b. You **don't need to create an array** first to use them.
- c. Here are the static method of arrays:

i. **Array.from():**

1. The **Array.from()** method creates a new array instance from an array-like object.
2. **Syntax:** Array.from(arrayLike);
3. **For Example** →

```
let str = "hello";
```

```
console.log(Array.from(str)); // (5)
['h', 'e', 'l', 'l', 'o']
```

ii. **Array.isArray():**

1. Checks if something is an array or not. Returns **true** or **false**.

2. **Syntax:** Array.isArray(arr)

3. **For Example** →

```
let arr = [1, 2, 3];
console.log(Array.isArray(arr)); // true
```

iii. **Array.of():**

1. Creates a **new array** from given values.

2. **Syntax:** Array.of(element1, element2, ...);

3. **For Example** →

```
let numArr = Array.of(1, 2, 3);
console.log(numArr); // (3) [1, 2, 3]
console.log(Array.isArray(numArr));
// true
```

5. Array Instance Methods:

- a. In JavaScript, array instance methods are functions that are built into the Array prototype, meaning they are available to all arrays.
- b. These methods allow you to manipulate, access, and perform operations on array elements.
- c. **Here are the list of array instance methods in JavaScript:**

i. **push():**

1. Adds one or more elements to the end of an array and returns the new length of the array.

2. **Syntax:**



```
array.push(element1, element2, ..., elementN)
```

3. **Return Type:** Number (new length array)

4. **For Example** →

```
let arr = [1, 2];  
arr.push(3);  
console.log(arr); // [1, 2, 3]
```

ii. **pop():**

1. Removes the last element from an array and returns it.

2. **Syntax:**



```
array.pop()
```

3. **Return Type:** The removed element

4. **For Example** →

```
let arr = [1, 2, 3];  
console.log(arr.pop()); // 3  
console.log(arr); // [1, 2]
```

iii. **shift():**

1. Removes the first element from an array and returns it.

2. **Syntax:**

 `array.shift()`

3. **Return Type:** The removed element

4. **For Example** →

```
let arr = [10, 20, 30];  
console.log(arr.shift()); // 10  
console.log(arr); // [20, 30]
```

iv. **unshift():**

1. Adds one or more elements to the beginning of an array and returns the new length.

2. **Syntax:**

 `array.unshift(element1, element2, ..., elementN)`

3. **Return Type:** **Number** (new length of the array)

4. **For Example** →

```
let arr = [2, 3];  
arr.unshift(1);  
console.log(arr); // [1, 2, 3]
```

v. **slice():**

1. Returns a shallow copy of a portion of an array without modifying the original.

2. **Syntax:**



`array.slice(startIndex, endIndex)`

3. **Return Type:** **Array** (new array with selected elements)

4. **For Example** →

```
let arr = [1, 2, 3, 4];  
console.log(arr.slice(1, 3)); // [2, 3]
```

vi. **splice():**

1. Adds, removes, or replaces elements in an array.

2. **Syntax:**



`array.splice(startIndex, deleteCount, item1, item2, ...)`

3. **Return Type:** **Array** (array of removed elements)

4. **For Example** →

```
let arr = [1, 2, 3, 4];  
arr.splice(1, 2);  
console.log(arr); // [1, 4]
```

vii. **concat():**

1. Merges two or more arrays into a new array.

2. **Syntax:**



`array.concat(array1, array2, ..., arrayN)`

3. **Return Type:** **Array** (new combined array)

4. **For Example** →

```
let a = [1, 2];  
let b = [3, 4];
```

```
console.log(a.concat(b)); // [1, 2, 3, 4]
```

viii. indexOf():

1. Returns the first index at which a given element is found, or **-1** if not present.

2. **Syntax:**



```
array.indexOf(searchElement, fromIndex)
```

3. **Return Type:** **Number** (index or **-1**)

4. **For Example** →

```
let arr = ["a", "b", "c"];  
console.log(arr.indexOf("b")); // 1
```

ix. includes():

1. Checks if an array contains a certain element.

2. **Syntax:**



```
array.includes(searchElement, fromIndex)
```

3. **Return Type:** **Boolean** (**true** or **false**)

4. **For Example** →

```
let arr = [1, 2, 3];  
console.log(arr.includes(2)); // true
```

x. reverse():

1. Reverses the order of elements **in place**.

2. **Syntax:**



`array.reverse()`

3. **Return Type:** Array (same array, reversed)

4. **For Example** →

```
let arr = [1, 2, 3];  
arr.reverse();  
console.log(arr); // [3, 2, 1]
```

xi. **join():**

1. Joins all array elements into a string with a separator.

2. **Syntax:**



`array.join(separator)`

3. **Return Type:** String

4. **For Example** →

```
let arr = ["a", "b", "c"];  
console.log(arr.join("-")); // "a-b-c"
```

xii. **keys():**

1. Returns an iterator with the array's keys (indexes).

2. **Syntax:**



`array.keys()`

3. **Return Type:** Array Iterator

4. **For Example** →

```
let arr = ["x", "y"];  
for (let key of arr.keys()) {
```

```
    console.log(key); // 0, 1
  }
```

xiii. **values():**

1. Returns an iterator with the array's values.

2. **Syntax:**

 `array.values()`

3. **Return Type:** Array Iterator

4. **For Example** →

```
let arr = ["x", "y"];
for (let val of arr.values()) {
  console.log(val); // "x", "y"
}
```

xiv. **flat():**

1. Flattens nested arrays into a single array (1 level by default).

2. **Syntax:**

 `array.flat(depth)`

3. **Return Type:** Array

4. **For Example** →

```
let arr = [1, [2, 3], [4, [5]]];
console.log(arr.flat()); // [1, 2, 3, 4,
[5]]
console.log(arr.flat(2)); // [1, 2, 3, 4, 5]
```

